TECHNICAL SPECIFICATION

Version: 2.0

MISSION FOR OPERATIONS & DEVELOPMENT

Office Address: MOD, C/B-037, Central Nongrim Hills, Shillong, East Khasi Hills, Meghalaya, India
Mob. +91 9046682303 | Email: straight2moders@gmail.com

Phase 1: Homepage

This section dictates the functional and visual requirements for the frontend accessed by unauthenticated users (Players). The entire UI must be engineered with a strict Mobile-First methodology to facilitate seamless WhatsApp routing.

Global UI Components (Throughout the Website)

A. **Design Philosophy:** The UI must visually be themed based on the logos of Housie Ghar. A modern themed and a User Interface that is easy to understand and use.

B. **Sticky Top Navigation:** Must remain anchored to the top of the viewport during scrolling.

Brand Logo: Positioned top left; clicking it returns the user to the home lobby.
Menu Items: Home, Games, Winners (Hall of Fame showing which user has won the greatest number of times), How to Play.
Staff Login: A discrete icon at the far-right routing to the secure staff (Superadmin/ admin/ operator/ bookie) login portal.

C. **Global Footer:** Must cleanly state "Powered by MOD".

1. The Landing Page (The Public Lobby)

The entry point for all players. It must load in under 2 seconds and instantly convey trust.

a) **The Hero Section:** A dynamic banner at the top of the viewport highlighting the most immediate upcoming "Game name set by Superadmin/admin" or displaying a live countdown timer to the next draw.

b) **The Game Cards Feed:** A vertically scrollable feed directly below the hero section, pulling real-time data from the Scheduled Games database table.

Core Details: Each card displays the Game Title, Date, Start Time, Ticket Price
Dynamic Progress Bar: A visual indicator calculating (Tickets Booked + Locked) / Total Tickets.
Urgency Badges: The UI automatically applies a "Fast Filling!" badge at 80% capacity and grays the card out with a "Sold Out" stamp at 100%.
Prize Pool: The exact monetary breakdown for winning categories (e.g. 1st Full House, Top Line, etc.)

2. The Game Room (Ticket Selection Grid)

Accessed by clicking an available Game Card. This is the primary conversion funnel.

a) **The Responsive Ticket Grid:** Based on the number of tickets set for the game, provide a grid of tables showing tickets, e.g. if 50 tickets are shown it should show numbers from 1-50 in the grid, these are just the ticket numbers. Selecting each number should then show the housie ticket below it for the user/player to see.

b) **State-Based Visual Rendering (Live WebSockets):** The grid connects via WebSockets or Server-Sent Events (SSE) to update ticket availability instantly without page refreshes.
Available: Clean background, clickable.
Locked: Yellow/Orange background, spinning loader icon, unclickable (indicating another user is currently paying).

Sold: Greyed out, strikethrough text, unclickable.

c) **The Sticky Action Footer:** Appears at the bottom of the screen the moment a player multi-selects available ticket.
Housie Name Input: A strictly mandatory text field for the player's nickname, complete with regex profanity filters. Copywriting here should encourage fun, Darjeeling/Sikkim localized nicknames to build community spirit.
Calculated Total & CTA: Displays the total payable amount alongside the primary "Book Now" button.

3. The Concurrency Engine & P2P Handoff

The critical transition phase where the platform secures the ticket and routes the player to the Agent.

a) **The Soft Lock Modal:** Upon clicking "Book Now," the backend performs a race-condition check. If available, a non-dismissible modal overlays the screen.
Visual Timer: A prominent countdown clock starting at 10:00.
Instructional Text: Advises the user that their tickets are reserved and asks the player to make the payment to their bookies within the time limit.

b) **The WhatsApp Redirect (wa.me Protocol):** Simultaneous to the modal, the browser triggers a deep link opening the native WhatsApp application.
Dynamic Routing: The backend assigns an Agent and pre-fills the chat with a structured message: "Hi! I am [Housie Name]. I want to book Ticket(s): [Ticket Numbers] for the [Game Title] at [Game Time]. Booking ID: #[Booking_ID].".

c) **Background Polling & Resolution:** The web tab remains open and polls the backend every 3 to 5 seconds. Once the Agent clicks "Confirm Payment" on their dashboard, the frontend detects the state change, destroys the timer, and triggers a green "Success" animation, displaying the digital tickets.

4. The Live Execution Board (The Automated Game)

The immersive interface activated when the scheduled game begins.

a) **Screen Wake Lock API:** The browser utilizes this API to prevent the smartphone screen from auto-dimming during the draw.

b) **The Housie (Tambola) Cage & Audio Tease:**

- The backend engine broadcasts drawn numbers via SSE.
- The frontend instantly plays the custom audio file featuring regional slang or dialect phrasing mapped to that specific number.
- *The Tease:* The visual display of the number on the digital board is intentionally delayed by roughly 1200ms after the audio starts to mimic the suspense of a physical caller.

c) **Auto-Marking Tickets:** The system automatically scans the player's digital tickets and applies a CSS highlight class to match drawn numbers, simulating a pen crossing them out.

d) **Live Winners Feed & Overlays:** When the automated engine detects a winning pattern (e.g., Quick 5, Full House), the gameplay pauses for roughly 4 seconds to trigger a celebratory overlay highlighting the winner's Housie Name and ticket. During a game a grid showing the prizes alongside the Housie Name and number

should also be shown, probably below the draw that's taking place above and the numbers that have been called so far being displayed. The names will automatically fill in the respective prize slot during the game.

e) **Live Emoji Reactions:** A floating reaction bar allows players to tap emojis that float up the side of the screen, replicating the cheers of a physical hall without the server strain of text chat.

Phase 2: The Secure Management

This section dictates the architectural and functional requirements for the unified, authenticated backend environment. All internal staff portals, Level 1 through Level 4, will operate within this secure domain, sharing a cohesive design system and UI language to ensure seamless navigation and connectivity across the organization.

Global UI & Authentication Flow

As described in Phase 1 for Global UI we will follow the same interface and theming.

A. Unified Design System & Empty States

Cohesive UI: All roles experience the same core layout, a fixed sidebar navigation menu on the left and a top status bar, ensuring all interfaces looks and feels like a native extension of the Housie Ghar Homepage.

Platform Initialization (Empty State): At the beginning of the platform's lifecycle, the database is completely empty (no games, no stats, no staff). The UI must handle these empty states gracefully, prompting users to "Create First Game" or “Add Staff.”

Filling Status Dashboard: A globally accessible widget or page for all authenticated staff (Levels 1 to 4) that allows them to check the real-time filling status (tickets booked/sold vs. capacity) of all scheduled games.

B. Security & The Onboarding Flow

Superadmin Default Genesis Login: The platform initializes with a single master account. The Superadmin logs in using the default ID: superadmin and Password: Enterhg@0902.

Credential Reset (OTP Verification): Upon initial login, or at any time later, the Superadmin (and all subsequent staff) can change their credentials by navigating to their profile, triggering an OTP (One-Time Password) sent to their registered mobile number or email ID, and setting a new username and secure password.

Staff Provisioning: The Superadmin and Admins manually create profiles for lower-tier staff, setting their initial login IDs and temporary passwords. When these staff log in for the first time, they follow the same OTP verification process to secure their accounts.

1. Level 1: The Superadmin Control Center (Ultimate Authority)

The Superadmin possesses unrestricted access across the entire platform, acting as the master controller and financial overseer.

a) Master Game Controls

- **Lifecycle Management:** Full authority to schedule new games, start, pause, or completely stop any live game.
- **Advanced Engine Overrides:** Exclusive ability to forcefully *reshuffle tickets* or *reset calls* (clearing the drawn number history) if a critical error occurs during a session or if a game needs to be restarted from scratch.

b) Workforce Provisioning

- Can create, edit, and delete profiles for Admins, Operators, and Bookies, assigning their initial login credentials.

c) Financial Hub & Analytics

- **Wallet Approvals:** Acts as the central bank, reviewing and accepting/rejecting digital wallet recharge requests submitted by Bookies. He can also assign which admin should instead receive these wallet requests from the Bookies and handle the financial aspects of the game. Once he chooses which admin can handle the financial aspects of the game, that admin's admin portal dashboard should further enhance and turn into an admin/financial officer dashboard to handle and manage our finances in an easier manner.
- **Master Dashboard:** Complete visibility into all financial stats (gross volume, profit distributions) and historical game stats.

2. Level 2: The Admin Dashboard

Admins share significant operational power with the Superadmin to keep the platform running efficiently without relying on a single bottleneck.

a) Game Management

- Can independently schedule new games.
- Maintains the authority to start, pause, or stop games as needed to maintain operational integrity.
- If the superadmin has chosen the particular admin to handle the wallet system and financial aspects of the game, the admin's dashboard UI and portal will enhance and turn into admin/CFO portal to manage all finances.

b) Workforce & Pipeline Management

- Authorized to provision and create new Operators and Bookies, supplying their initial login IDs and passwords.

- **Wallet Approvals:** Shares the ability to accept and process digital wallet recharge requests directly from Bookies to keep the sales pipeline moving.

2.1 The Admin/Financial Officer (CFO) Dashboard

Transformation Trigger: When the Superadmin designates a specific Admin as the Financial Officer, their standard Level 2 interface dynamically unlocks a highly secure, specialized **Financial Hub** routing module. This portal shifts the Admin's focus from game creation to pure liquidity and ledger management.

A. UI/UX & Layout Enhancements

- **The Financial HUD (Heads-Up Display):** The standard top status bar expands into a sticky financial ribbon displaying real-time monetary health: *Total Platform Liability* (the sum of all active Bookie wallet balances), *Daily Gross Processed*, and a flashing counter for *Pending Recharge Requests*.
- **Split-Screen Queue Interface:** To support the high-speed nature of WhatsApp transactions, the primary workspace shifts to a dual-pane layout. The left pane lists incoming requests, and the right pane displays the specific Bookie's historical ledger and trust score.
- **Action-Oriented Color Coding:** Financial buttons utilize strict, high-contrast color coding (e.g., vibrant green for "Credit Wallet," stark red for "Reject/Dispute") to prevent costly misclicks during rapid, high-volume processing.

B. Core Functions & Responsibilities

- **Direct WhatsApp Sync & Approval Engine:** Because Bookies are redirected to the FO's WhatsApp to request funds, the dashboard features a live queue of these pending requests. Once the FO visually verifies the fiat deposit in their banking app, they click "Approve" on the corresponding dashboard card. This instantly executes an ACID-compliant transaction to credit the Bookie's digital wallet.

- **The Master Bookie Ledger:** A filterable, sortable data table providing a macro-view of the entire sales force. It displays every Bookie's Current Balance, Total Lifetime Top-Ups, and Last Recharge Timestamp.

- **Low-Balance Proactive Alerts:** The system automatically highlights Bookies whose digital wallets drop below a critical threshold (e.g., ₹500). The FO can use a 1-click WhatsApp template to proactively message them: *"Your inventory is low. Top up before the 8:00 PM game to ensure you don't miss sales."*

- **Manual Overrides & Dispute Resolution:** The FO has the authority to perform manual ledger adjustments (credits or debits) outside the standard request flow to correct technical glitches or bounced physical transactions. Every manual adjustment mandates the completion of a "Reason/Reference" text field to preserve an immutable audit trail for the Superadmin.

- **End-of-Day (EOD) Reconciliation:** An automated reporting module that compiles the day's financial velocity. It cross-references the total digital inventory issued to Bookies against the gross ticket sales, generating a clean reconciliation sheet to ensure zero leakage between physical bank deposits and digital platform assets.

3. Level 3: The Operator Console

Operators have a focused operational scope, primarily responsible for the creation and real-time execution of the Housie events.

a) Game Execution

- Authorized to schedule new games under their jurisdiction.
- Can start, pause, and stop games directly from their live execution HUD.

b) Status Monitoring

- Full access to monitor the filling status of all scheduled games, allowing them to track ticket sales momentum leading up to the draw they are hosting.

4. Level 4: The Bookie Workspace

Fully integrated into the unified backend architecture, Bookies (Agents) manage the decentralized point-of-sale and act as the platform's financial frontline.

a) Digital Wallet Management

- **Recharge Engine:** Bookies can submit wallet recharge requests to the Admins or Superadmin when their digital inventory runs low, attaching necessary transfer references. Once he hits request for funds it will redirect him to the WhatsApp of the particular admin who has been set as Financial Officer by the Superadmin.

b) The Booking Queue (WhatsApp Fulfillment)

- While logging in for the first time, the staffs will be asked to complete their profile and every detail should be filled by them including their whatsapp number, etc.
- Bookies receive incoming booking requests generated via the frontend WhatsApp redirect.
- All Bookies should receive booking requests in a turn-by-turn manner. For example: If there are 5 Bookies, the first booking request for any game should go to Bookie 1, the second booking request goes to Bookie 2 and so on. This will provide fair booking chances to all bookies and proper handling of bookings as well, so that no double bookings of the same number can take place as well.
- **The 10-Minute Lock:** The system initiates a 10-minute countdown timer the moment a player requests a ticket.
- **Resolution:**
  Confirm: Upon receiving the fiat payment, the Bookie clicks "Confirm," deducting the amount from their wallet and locking the ticket for the user.
  Cancel: If no payment is received before the 10-minute timer expires, the Bookie can cancel the ticket (or the system auto-cancels it), releasing it back to the public pool.

- **Dynamic Wallet Validation (The Smart Queue):** When a player clicks "Book Now," the backend calculates the Total Payable Amount of the requested tickets. Before assigning the booking and generating the WhatsApp redirect link, the system queries the database for the Current Wallet Balance of the Bookie whose turn is next in the Round-Robin queue.
- **The Skip-and-Route Logic:**
- If the active Bookie's balance is greater than or equal to the Total Payable Amount, the system assigns the booking to them and generates the redirect link.
- If the active Bookie's balance is less than the Total Payable Amount, the backend instantly skips them and queries the next Bookie in the sequence. It continues this microsecond loop until it finds a Bookie with sufficient inventory to fulfill the specific order.
- **Proactive "FOMO" Alerts (Agent Motivation):** When a Bookie gets skipped by the algorithm, the backend instantly triggers a UI notification on their dashboard (and logs it in the FO portal) stating: *"Alert: You just missed a booking request because your wallet balance is too low. Recharge immediately to resume receiving sales."* This utilizes the Fear Of Missing Out (FOMO) to organically pressure Bookies into keeping their digital wallets well-funded.

The Liquidity-Aware Routing Engine: When a player initiates a booking, the system calculates the Total Payable Amount and scans the active Bookie queue in a Round-Robin sequence.

- If a Bookie's Current Wallet Balance is less than the payable amount, the booking is routed to their WhatsApp.
- If a Bookie lacks funds, the system skips them (triggering a "FOMO" alert on their dashboard) and instantly checks the next Bookie.

The Operator Overflow Failsafe: In the rare event that the system completes a full loop of the active Bookie queue and finds that *zero* Bookies have sufficient digital inventory:

- **Fallback Routing:** The backend bypasses the Bookie tier entirely and dynamically generates the WhatsApp wa.me redirect link for the specific **Operator** assigned to host that game.
- **Manual Fulfillment Flow:** The player's pre-filled message lands directly in the Operator's WhatsApp. The Operator acts as the ultimate backstop, manually sending their personal/MOD UPI QR code or ID to the player.

Operator Dashboard Enhancement (The Fallback Queue):

- To support this failsafe, the Level 3 Operator Console must feature a specialized "Overflow Booking Queue" tab.
- When the system routes a player to the Operator, the corresponding Request Card appears in this tab.
- Once the Operator verifies the fiat payment in their banking app, they click "Force Confirm." Because the Operator is an internal staff member managing the game, this action locks the ticket for the player *without* requiring a digital wallet deduction, essentially acting as a direct-to-platform sale.

c) Status Monitoring

- Bookies can track the real-time filling status of all scheduled games, allowing them to anticipate high-volume periods or push marketing to their WhatsApp groups for games that are struggling to fill.

Phase 3: Technical Infrastructure

With the User Interface (Phase 1) and the secure unified Management (Phase 2) fully mapped out, Phase 3 defines the core technical heartbeat of Housie Ghar. This section details the backend algorithms, the real-time execution engine, and the server architecture required to ensure the platform operates with zero human intervention during a draw, handles massive concurrency, and maintains absolute financial integrity.

1. Game State Management & The RNG Protocol

To build player trust and ensure auditability, the platform must rely on strict cryptographic standards rather than basic randomization scripts.

a) Immutable State Machine A game instance progresses through strict database states that enforce logic restrictions:

- **Scheduled:** Admins can edit parameters; Bookies can lock tickets.
- **Live:** Triggered by the Operator. Booking is instantly locked; WebSockets begin broadcasting.
- **Paused:** Emergency state halting the draw sequence.
- **Completed:** Triggered autonomously once all prizes are claimed.

b) The RNG (Random Number Generator) Engine

- **Cryptographic Security:** The backend must utilize a Cryptographically Secure Pseudorandom Number Generator (CSPRNG), such as crypto.randomBytes or crypto.randomInt in Node.js. Standard Math.random() is strictly prohibited.
- **The Pre-Draw Shuffle Array:** When the Operator clicks "Start Game," the backend generates an array of numbers [1–90] and applies a secure Fisher-Yates shuffle.
- **The Audit Trail:** This exact shuffled sequence is immediately saved to the Game_Logs table under a draw_sequence column before the first number is even broadcast. This proves no mid-game manipulation occurred and allows the system to perfectly resume if a server crashes.

2. The Conductor (Pacing & Execution)

The Conductor is the backend chronometer responsible for popping numbers from the array and synchronizing the frontend experience.

a) Dynamic Speed Control

- The system utilizes a configurable call_interval variable.

- When the Operator moves the "Speed Slider" on their Level 3 dashboard (e.g., from 8 seconds down to 5 seconds), the backend Conductor intercepts this new value and applies it immediately to the next draw interval without dropping the live connection.

b) Audio-Visual Synchronization Flow

- The backend broadcasts a JSON payload via Server-Sent Events (SSE) every tick: { "draw_number": 42, "total_drawn": 15, "timestamp": "..." }.
- The frontend intercepts this, instantly plays the localized .mp3 file, and executes the 1.2-second "Tease" delay before visually displaying the number on the Tambola Cage.

3. The Automated Win Detection Engine

This is the most computationally intensive module. The backend must evaluate thousands of tickets for multiple winning patterns within milliseconds of every single draw.

a) Pattern Evaluation Logic A ticket is stored in the database as a structured 3x9 matrix array (e.g., Row 1: [null, 12, null, 34...]). Every time a new number is pushed to the drawn_numbers array, the engine evaluates:

- **Quick 5:** Does the intersection of the ticket's numbers and the drawn numbers equal 5?
- **Lines (Top/Middle/Bottom):** Are all 5 non-null values in the specific row present in the drawn numbers?
- **Full House:** Are all 15 non-null values across all rows present?

b) Resolution & Tie-Breaker Logic

- **Locking the Prize:** Once a pattern is matched, the engine marks that prize as "Claimed" in the database so it is no longer evaluated.
- **Simultaneous Wins (Split Logic):** If two tickets achieve a Full House on the exact same drawn number, the backend detects an array of winners > 1. The engine automatically splits the monetary prize equally, logs it, and broadcasts both winning Housie Names simultaneously to the frontend.

4. Concurrency, Database Architecture, & Scalability

The infrastructure must be engineered to handle massive, simultaneous traffic spikes (e.g., thousands of players checking the game board at exactly 7:59 PM for an 8:00 PM game) without double-booking tickets or crashing.

a) ACID Compliance & Race Condition Prevention

- **Database Row Locking:** When a user triggers the "Book Now" action, the backend executes an explicit SELECT FOR UPDATE query on the PostgreSQL Tickets table. This places a pessimistic lock on those rows, physically preventing a second player from booking the same ticket 10 milliseconds later.
- **Transactional Integrity:** The Bookie's "Confirm Payment" action and the CFO's "Wallet Credit" action are wrapped in strict ACID transactions (Begin $\rightarrow$ Check Balance $\rightarrow$ Deduct $\rightarrow$ Update Status $\rightarrow$ Commit). If any step fails, the entire transaction rolls back.

b) Real-Time Infrastructure

- **SSE vs. WebSockets:** Server-Sent Events (SSE) are utilized for the massive Player UI, as data flows strictly one-way (Server $\rightarrow$ Client), making it vastly more memory-efficient than maintaining thousands of two-way WebSockets. Standard WebSockets are reserved for the Admin/Operator/Bookie dashboards where two-way actions occur.
- **Redis Pub/Sub:** A central Redis cluster acts as the message broker. If the platform scales to multiple servers, the main Game Engine publishes the drawn number to Redis, which instantly relays it across all server nodes so every connected player receives the update simultaneously.

c) Security & Failsafes

- **Stateless Security:** All authentication utilizes JSON Web Tokens (JWT) stored in secure, HttpOnly cookies to prevent Cross-Site Scripting (XSS).
- **Rate Limiting:** The public booking API is protected by strict IP-based rate limiting (e.g., maximum 5 locking attempts per minute) and Cloudflare to prevent automated bot scraping or DDoS attacks.
- **"Ghost Host" Auto-Resume:** If the backend Node.js server crashes mid-game, it reboots, checks the database for game_status == 'Live', reads the previously saved draw_sequence, and seamlessly resumes the game autonomously.

5. Server Scalability & Load Balancing

The architecture must be highly elastic to accommodate massive, temporary traffic spikes leading up to a scheduled draw.

- **Stateless Backend Design:** The Node.js application servers must be entirely stateless. All session data, game states, and countdown timers must live in the Redis cluster rather than the server's local RAM, allowing the load balancer to route any request to any active server seamlessly.
- **Auto-Scaling Groups (ASG):** The backend infrastructure should be deployed behind an Application Load Balancer (ALB). If CPU utilization hits 70% exactly 10 minutes before a game, the ASG will automatically spin up new server instances to handle the incoming rush of players, and scale back down after the game ends to optimize cloud hosting costs.

6. Disaster Recovery (DR) & Backups

In the event of a catastrophic database failure, the system must recover without losing a single financial record or ticket asset.

- **Automated Daily Snapshots:** The primary PostgreSQL database must execute automated, full backups daily at 3:00 AM IST, during the platform's lowest traffic period.
- **Point-in-Time Recovery (PITR):** The database cluster must retain Write-Ahead Logs (WAL). This allows the Superadmin to restore the entire platform to a specific minute in time (e.g., restoring exactly to 8:14 PM if a critical data corruption occurred at 8:15 PM).

7. Reconnection Logic & State Hydration

Because the primary user base relies on mobile networks, the system must gracefully handle sudden 4G/5G connection drops.

- **Auto-Reconnection:** If a player drives through a tunnel or loses service during a live draw, their browser must automatically attempt to reconnect to the SSE stream.
- **State Hydration:** Upon successful reconnection, the frontend must instantly request the current drawn_numbers array and the claimed_prizes list. This allows the UI to re-sync their ticket visuals instantly, ensuring they haven't missed a single number call.

8. The Recommended Technology Stack

For clarity to the development team, the finalized tech stack executing Phase 1, 2, and 3 is strictly defined as:

- **Frontend:** Next.js (React) or Nuxt.js (Vue) styled with Tailwind CSS for mobile-first responsiveness.
- **Backend:** Node.js (Express/NestJS) or Python (FastAPI) to handle high-concurrency event loops.
- **Primary Database:** PostgreSQL (strictly relational for ACID financial compliance).
- **In-Memory Cache:** Redis (for Pub/Sub WebSocket broadcasting and state management).

9. Data Security: Encryption & Compliance

While we covered endpoint security (JWTs and Rate Limiting), the platform handles sensitive P2P transaction data and session details that require strict encryption protocols.

- **Encryption in Transit:** All data transmissions, especially Agent/Bookie banking details, UPI IDs, and Player session data, must strictly occur over TLS 1.3 (HTTPS).

- **Encryption at Rest:** The primary PostgreSQL database must be encrypted at rest utilizing standard cloud provider management keys (e.g., AWS KMS) to ensure data integrity even if the physical servers are compromised.

10. The Ticket Data Structure (Database Level)

To ensure the Automated Win Detection Engine can run its evaluations within milliseconds, developers must not store tickets as simple images or flat strings.

- **JSON/Matrix Format:** A Housie ticket is a 3x9 grid. The database must store each ticket as a structured JSON object or matrix array.
- *Example Structure:*

  Row 1: [null, 12, null, 34, 45, null, 61, null, 88]
  Row 2: [3, null, 25, null, 48, 55, null, 72, null]
  Row 3: [7, 18, null, 39, null, null, 68, 77, null]

- This exact structure allows the Node.js or Python backend to perform instantaneous array intersection checks against the drawn numbers list without heavy processing overhead.

11. Technical Accessibility & Edge Cases

There are a couple of final technical hooks required on the frontend to handle specific user environments.

- **Color Blindness Fallbacks:** The UI must not rely on color alone to convey state. For example, a "Locked" ticket shouldn't just turn yellow; it must functionally display a "Lock" icon. A "Sold" ticket must have a strikethrough or an "X".
- **Audio Mute Toggle:** A prominent, easily accessible mute button must be engineered into the live game screen for players in quiet environments, allowing them to disable the automated audio voiceovers while retaining the visual SSE number updates.

12. Strict Technical Exclusions (Out of Scope)

To guarantee a streamlined development cycle, rapid deployment, and prevent scope creep, the developers must strictly adhere to the following exclusions for this phase of development:

- **No Native Applications:** Native iOS or Android application development is completely out of scope; the platform must function flawlessly purely as a mobile-responsive web application.
- **No Internal Fiat Wallets for Players:** The system will not host direct, in-app fiat wallets for Players. Players will solely use external UPI apps for the P2P transfers.

- **No Live Video Streaming:** There will be no integrations for live video streaming (e.g., streaming a webcam feed of a physical host). All live gameplay must rely entirely on the platform's automated UI animations and localized audio triggers.

13. Low-Bandwidth Asset Optimization

Recognizing that players will access the game from areas with fluctuating 3G/4G network conditions, the frontend asset delivery must be ruthlessly optimized.

- **CSS vs. Heavy Media:** Large, heavy background images must be heavily compressed or avoided entirely. The developers must exclusively use lightweight CSS-based animations for the Tambola cage and win celebrations.
- **Media Restrictions:** The use of heavy video files or GIFs for gameplay animations is strictly prohibited to prevent latency and server strain.

Phase 4: Database Schema

This phase maps out the exact PostgreSQL relational database structures required to support the platform's concurrency, financial tracking, and automated live game engine. A strict Relational Database Management System (RDBMS) is mandatory to enforce ACID compliance across all P2P transactions and ticket bookings.

1. Users **Table (Staff & RBAC)**

This table stores all authenticated internal workforce accounts (Level 1 through Level 4). The system strictly queries this table to validate access roles and digital inventory capacity.

Column Name	Data Type	Description
user_id	UUID (Primary Key)	Unique identifier for the staff member.
username	VARCHAR	Used for dashboard authentication.
password_hash	VARCHAR	Securely hashed password.
role_id	INT	Defines hierarchy (1=Superadmin, 2=Admin, 3=Operator, 4=Bookie).
is_cfo	BOOLEAN	Flag set by Superadmin indicating if an Admin handles finances.
whatsapp_number	VARCHAR	Mandatory contact number used for routing P2P payment requests.
current_balance	DECIMAL	The Bookie's active digital wallet inventory value (Default: 0.00).
status	VARCHAR	Active, Suspended, or Pending Registration.

2. Scheduled_Games **Table**

The central record for all Housie events, controlling the automated engine's logic and the frontend UI's state.

Column Name	Data Type	Description
game_id	UUID (Primary Key)	Unique identifier for the game.
game_title	VARCHAR	e.g., "Sunday Mega Draw".
scheduled_time	TIMESTAMP	The exact date and time the game is set to begin.
ticket_capacity	INT	Total number of tickets generated for this instance (e.g., 500).
ticket_price	DECIMAL	Base cost per ticket.
operator_id	UUID (Foreign Key)	Links to the Users table (Level 3 Operator hosting the game).
game_status	VARCHAR	'Scheduled', 'Live', 'Paused', 'Completed', or 'Postponed'.
call_interval	INT	The dynamic speed variable in milliseconds (e.g., 8000ms).
prize_pool_config	JSONB	Contains the active winning categories and their monetary values.

3. Tickets **Table**

This is the most highly trafficked table. It requires pessimistic row locking (SELECT FOR UPDATE) to prevent race conditions when multiple players attempt to book simultaneously.

Column Name	Data Type	Description
ticket_id	UUID (Primary Key)	Unique identifier for the specific ticket.
game_id	UUID (Foreign Key)	Links to Scheduled_Games.

Column Name	Data Type	Description
matrix_structure	JSONB	The 3x9 grid matrix (e.g., [[null, 12...], [3, null...]]) used for rapid automated win detection.
status	VARCHAR	'Available', 'Locked', or 'Sold'.
booking_id	VARCHAR	Generated temporarily during the soft-lock phase.
player_housie_name	VARCHAR	The mandatory, localized nickname inputted by the player.
locked_until	TIMESTAMP	The 10-minute expiry timestamp for pending P2P payments.
assigned_bookie_id	UUID (Foreign Key)	The Bookie assigned to fulfill this specific ticket via the Round-Robin algorithm.

4. Wallet_Ledger **Table**

An immutable tracking log for all financial events, acting as the internal accounting system for Bookies and the CFO.

Column Name	Data Type	Description
transaction_id	UUID (Primary Key)	Unique identifier for the ledger entry.
bookie_id	UUID (Foreign Key)	Links to the specific Agent/Bookie.
transaction_type	VARCHAR	'Credit' (Top-Up) or 'Debit' (Booking Confirmation).
amount	DECIMAL	The monetary value of the transaction.
reference_note	VARCHAR	Associated booking_id for debits, or bank reference for credits.
status	VARCHAR	'Pending' (waiting for CFO approval) or 'Approved'.

Column Name	Data Type	Description
timestamp	TIMESTAMP	Exact time the transaction occurred or was requested.

5. Game_Logs **(The Engine Audit Trail)**

This table acts as the failsafe recovery system and audit trail for the Automated Game Engine.

Column Name	Data Type	Description
log_id	UUID (Primary Key)	Unique identifier for the log.
game_id	UUID (Foreign Key)	Links to Scheduled_Games.
draw_sequence	JSONB	The pre-generated, cryptographically shuffled array of numbers 1-90.
drawn_numbers	JSONB	An actively updating array of numbers that have already been called.
claimed_prizes	JSONB	Records which prizes (e.g., Quick 5, Full House) have been won, including the winner's details and tie-breaker splits.

While we have mapped out the core transactional and gameplay architecture, there are three final support tables required to make Phase 4 completely aligned with the organizational oversight and localization requirements defined in the blueprint.

To ensure the Superadmin has absolute audit visibility and can dynamically update the platform without developer intervention, we must append the following tables to the schema:

6. System_Audit_Logs **(The Master Audit Trail)**

Trust requires strict accountability. This table serves as the immutable, chronological ledger recording every action taken by the staff on the backend to prevent internal fraud or unauthorized edits.

Column Name	Data Type	Description
log_id	UUID (Primary Key)	Unique identifier for the audit entry.
timestamp	TIMESTAMP	The exact microsecond the action occurred.
user_id	UUID (Foreign Key)	Links to the Users table (the staff member performing the action).
action_performed	VARCHAR	e.g., "Created Game #1042", "Suspended Agent", "Approved Top-Up".
target_data	VARCHAR	The specific ID of the game, user, or ledger entry affected.
ip_address	VARCHAR	The network IP address of the staff member for security tracking.

7. Global_Configurations **(Theming & Variables)**

This table allows the Superadmin to instantly adapt the platform to different seasons or marketing pushes without requiring developers to push new code to production.

Column Name	Data Type	Description
config_key	VARCHAR (Primary Key)	e.g., active_theme, support_email, marquee_text, terms_text.
config_value	TEXT	The assigned value (e.g., 'Dark Mode', 'info@housieghar.com').
last_updated_by	UUID (Foreign Key)	The Superadmin who last modified the variable.
updated_at	TIMESTAMP	When the configuration was changed.

8. Audio_Assets_Mapping **(Cultural Localization)**

To ensure the game feels authentic and localized, this table maps the 90 distinct numbers to their specific regional audio files.

Column Name	Data Type	Description
number_id	INT (Primary Key)	The Housie number (1 through 90).
file_url	VARCHAR	The storage path to the .mp3 or .wav file (max 500KB) featuring the local dialect.
uploaded_by	UUID (Foreign Key)	The Superadmin who uploaded or updated the asset.

Phase 5: Real-Time Data Streams

This phase translates the platform's business logic, P2P routing, and live game mechanics into strict, actionable endpoints for the development team. It defines the core RESTful APIs and the Server-Sent Events (SSE)/WebSocket channels required to power Housie Ghar securely.

1. Authentication & Onboarding APIs

These endpoints handle the Zero-Trust security model, utilizing HttpOnly JSON Web Tokens (JWT) for all staff portals.

- POST /api/auth/login
  - **Description:** Accepts initial credentials (e.g., the Superadmin genesis login or a Bookie's temporary password).
  - **Action:** If successful, triggers the OTP flow and returns an otp_session_token.
- POST /api/auth/verify-otp
  - **Description:** Validates the OTP sent to the user's mobile/email.
  - **Action:** Issues the secure JWT. If it is the user's first login, it flags the frontend to forcefully route them to the Mandatory Profile Completion screen to register their WhatsApp number.

2. The P2P Booking Engine APIs (Frontend to Backend)

These routes handle the most critical, high-traffic conversion funnel: securing a ticket and routing the player to the Bookie or Operator.

- POST /api/booking/lock-ticket
  - **Security:** Highly rate-limited (e.g., max 5 attempts per minute per IP) to prevent bot scraping.
  - **Process:**
    1. Executes a SELECT FOR UPDATE to lock the requested ticket_ids.
    2. Validates the mandatory housie_name (e.g., "Priyush_99") against profanity regex.
    3. Runs the **Liquidity-Aware Round-Robin**: Checks the next Bookie's wallet balance. If insufficient, skips to the next Bookie. If all Bookies fail, routes to the Level 3 Operator (Overflow Failsafe).
  - **Response Payload:** Returns the booking_id, the locked_until timestamp (10 minutes), and the dynamically generated wa.me WhatsApp redirect URL.
- GET /api/booking/status/[booking_id]
  - **Description:** The background polling endpoint the frontend pings every 3 to 5 seconds while the player is completing the WhatsApp payment.
  - **Response:** Returns the current state (Locked, Sold, or Cancelled). If Sold, the frontend destroys the countdown timer and displays the digital tickets.

3. Financial Ledger & Wallet APIs

These endpoints manage the secure flow of digital inventory between the central platform (CFO/Superadmin) and the decentralized Bookies.

- POST /api/wallet/request-recharge
  - **Description:** Submitted by a Bookie when digital inventory is low.
  - **Action:** Registers the pending transaction in the Wallet_Ledger table and returns the CFO's wa.me URL so the Bookie can send the fiat transfer receipt.
- POST /api/wallet/approve-recharge
  - **Description:** Executed exclusively by the CFO or Superadmin after verifying the bank deposit.
  - **Action:** Wraps a strict ACID transaction to credit the Bookie's current_balance and update the ledger status to Approved.

4. Game Management & Execution APIs

Routes utilized by the Level 2 Admins and Level 3 Operators to orchestrate the Housie events.

- POST /api/games/create
  - **Description:** Creates a new game instance.
  - **Payload:** Accepts the scheduled date, capacity, ticket price, and the prize_pool_config object. The backend rejects the request if the prize pool exceeds 80% of the potential gross.
- PUT /api/games/[game_id]/status
  - **Description:** Updates the immutable state machine of the game.
  - **Action:** When updated to Live, the backend generates the CSPRNG Fisher-Yates array, saves it to draw_sequence, and locks all further ticket bookings.

5. Real-Time Infrastructure (SSE & WebSockets)

Traditional HTTP requests are too slow for the live draw. This section dictates the real-time data streams.

- GET /api/stream/game/[game_id] **(Server-Sent Events)**
  - **Description:** A one-way, highly memory-efficient data stream pushing the automated draw updates from the backend Conductor to thousands of connected players.
  - **Tick Payload:** Emitted based on the Operator's dynamic call_interval setting:

JSON

```
{
 "draw_number": 42,
 "total_drawn": 15,
```

```
 "timestamp": "1717859041234"
}
```

* **The Winner Broadcast Event**[cite: 1]
  * **Description:** Fired globally to all connected clients the millisecond the automated engine detects a winning matrix pattern (Quick 5, Lines, Full House)[cite: 1].
  * **Payload:**

```json
{
  "event": "winner",
  "prize": "Top Line",
  "housie_name": "Priyush_99",
  "ticket_id": 105,
  "amount": 1000
}
```

6. Bi-Directional WebSockets (Staff Portals)

While the Player UI uses one-way SSE for memory efficiency, the Admin, Operator, and Bookie dashboards require standard bi-directional WebSockets (e.g., Socket.io) to instantly push and pull operational commands without page refreshes.

- **The Bookie Live Queue Stream**
  - **Description:** An active WebSocket connection pushing new booking_id requests directly to the assigned Bookie.
  - **Action:** When a player locks a ticket, this stream instantly renders the Request Card on the Bookie's dashboard and triggers a subtle audio ping.
- **The Operator Control Stream**
  - **Description:** The two-way pipeline for the Operator hosting the live game.
  - **Actions:** Transmits the "Emergency Pause" and "Resume" commands directly to the backend Conductor, and instantly relays the new call_interval value when the Operator adjusts the dynamic speed slider.

7. Client State Hydration API

Because mobile network drops are inevitable, the platform needs a dedicated route for rapid UI recovery.

- GET /api/game/[game_id]/sync
  - **Description:** Triggered automatically by the frontend if a player's browser loses and then regains connection to the SSE stream.
  - **Response Payload:** The backend instantly serves the complete, up-to-date drawn_numbers array and the claimed_prizes list. This ensures the frontend immediately re-marks the player's tickets so they haven't missed a beat.

8. Interactive Smart Features API

This handles the lightweight social interactions designed to replicate the hall experience.

- POST /api/game/[game_id]/reaction
  - **Description:** The endpoint triggered when a player taps an emoji on the Live Emoji Reaction bar.
  - **Action:** To prevent server strain, this is a fire-and-forget payload that the backend validates and rapidly broadcasts to all connected clients via the SSE stream, rendering floating emojis on all screens without the overhead of a text chatroom.

9. Superadmin Emergency & Audit APIs

The ultimate failsafe routes reserved exclusively for Level 1 access to override standard platform behavior.

- POST /api/admin/emergency/override-game
  - **Description:** Accessed via the Superadmin's Emergency Console.
  - **Action:** Allows the Superadmin to force-pause the RNG Conductor, manually push a specific number to the frontend, or forcefully terminate a live game.
- POST /api/admin/emergency/bulk-refund
  - **Description:** Triggered if a game is forcefully terminated or severely compromised.
  - **Action:** Executes an automated script that voids the game's locked/sold tickets and systematically reverts the ticket values back to the respective Bookies' digital wallets.

Phase 6: UI/UX Guidelines & Smart Features

This final phase consolidates the design philosophy, strict localization requirements, and the definitive index of "Smart Features" that elevate the platform from a functional utility to an engaging, community-driven ecosystem. This section directly mirrors Chapter 9 of the blueprint to ensure developers understand the exact *feel* and accessibility standards required.

1. Core Design Philosophy

The UI must visually communicate excitement, high energy, and modern digital entertainment. The design will draw directly from the high-contrast, comic-explosion, and neon-badge styles of the official logos to create an immersive, arcade-like feel.

A. The Color Palette (Strict Mapping)

To make the brand colors "pop" exactly as they do in the logos, the entire platform must be built on a **Dark Mode Default** foundation.

- **Background Base:** Deep Black (#000000 to #111111) to emulate the backdrop of both logos.
- **Primary Brand Colors:**

  Neon Pink (sampled from the "HOUSIE" typography). Used for primary headers, active states, and celebratory win overlays.

  Electric Cyan/Teal (sampled from the "GHAR" script). Used for secondary buttons, active ticket highlights, and live UI accents.

- **Action & Accent Colors:**

  Vibrant Yellow/Gold (sampled from the tambola cage and pop-art explosion). Strictly reserved for Call-to-Action (CTA) buttons (e.g., "Book Now", "Confirm"), urgency badges ("Fast Filling!"), and the drawing animations.

  Crisp White & Bright Green (from the logo leaves/dice). Used sparingly for text readability and success state indicators.

B. Typography & Shapes

- **Headers & Titles:** Heavy, bold, and slightly retro sans-serif fonts to match the blocky, impactful nature of the "HOUSIE" text in Logo 1. Drop-shadows (as seen in Logo 2) should be utilized for primary page titles to create depth.

- **UI Components:** Buttons and game cards should feature sharp, modern borders, potentially incorporating the subtle, angled "action lines" or star motifs found in the explosion backdrop of Logo 2 for hover states.
- **The Game Board (Readability Override):** While the surrounding UI is energetic and highly stylized, the 1–90 ticket grids and the live draw numbers must remain clean, utilizing a high-legibility monospaced font in high contrast (e.g., White or Cyan on Dark Grey) to ensure users on small, low-resolution mobile screens never misread a number.

C. Animation Style

- **Pop-Art Celebrations:** When a player wins a prize (Quick 5, Full House), the UI overlay should mimic the yellow "comic book explosion" from HG Logo 2.jpg, bursting onto the screen with the floating pink and yellow stars before fading out.
- **The Tambola Cage:** The visual representation of the drawing engine can be modeled directly after the geometric, gold-line-art Tambola cage featured in the center of HG Logo 1.jpg.

2. Consolidated "Smart Features" Index

A centralized checklist for developers to ensure these specific quality-of-life enhancements are integrated across the respective modules:

a) **The WhatsApp 1-Click Template (Bookie UI):** A button on the pending request card that instantly copies a pre-formatted payment request to the device clipboard for rapid WhatsApp pasting.
b) **The "Ghost Host" Auto-Resume (Backend/Operator UI):** If the Operator's internet disconnects, the backend Conductor script ignores it and continues drawing numbers and awarding prizes autonomously.
c) **Live Emoji Reactions (Player UI):** A floating reaction bar allowing players to tap emojis that float up the screen, replicating physical hall cheers without the server strain of text chat.
d) **The Dynamic Speed Slider (Operator UI):** The ability for the host to slide the interval delay from 5 seconds to 12 seconds in real-time, instantly adapting to player feedback.
e) **The "Tease" Animation (Player UI):** During the live draw, the audio file plays first, followed by a 1.2-second delay before the visual number appears on the screen, mimicking the suspense of a physical caller.

3. Accessibility & Edge Cases

The frontend must be engineered to support all users and environments flawlessly.

- **Color Blindness Support:** The UI must never rely on color alone to convey status. For instance, a "Locked" ticket must display a "Lock" icon, and a "Sold" ticket must feature a strikethrough or an "X".

- **Screen Wake Lock API:** During the live game, the browser must utilize this API to prevent the player's smartphone screen from auto-dimming or going to sleep while they monitor their tickets.
- **Audio Mute Toggle:** A prominent, easily accessible mute button must be placed on the live game screen for players operating in quiet environments who prefer visual-only cues.

4. MOD Branding

While Housie Ghar features a high-energy, neon-pop consumer face, the underlying organizational credibility of MOD must be subtly but firmly established to build player trust.

- **Global Footer Integration:** The footer across all public and staff pages must cleanly and professionally state: *"Powered by MOD"*. This ensures the organizational authority is always present.
- **Strategic Trust Badges:** The landing page and the booking funnel must feature UI trust badges. Rather than generic "secure checkout" icons, these badges must specifically highlight the platform's unique selling points: the decentralized, zero-commission P2P payment structure and a "Fair Play" badge guaranteeing the automated, cryptographic RNG draws.